A Lightweight Retrieval-Augmented Shakespeare Assistant: Domain Adaptation with a Small Language Model

Group Number: *Insert Group Number*
CSCI433/933 Machine Learning Algorithms and Applications
Student Names and Student Numbers:
Cooper Kawari Menget (9179379), Bishal Bhatta (9990938), Raúl Palomo Mazo (1016374), Ping-Hsien Lo (1447865)

***Abstract*—We present a Shakespeare-aware question-answering system built around a small, deployable retrieval-augmented generation (RAG) pipeline. Three plays (*Hamlet*, *Macbeth*, and *Romeo and Juliet*) are indexed at the scene level using `sentence-transformers/all-MiniLM-L6-v2` embeddings; `phi3:3.8b` is called locally via Ollama to compose beginner-friendly answers conditioned on the retrieved scenes. A deterministic extractive composer is retained as a fallback when Ollama is not running, and a stylised composer produces short Shakespearean-style verse. The system runs end-to-end on a standard laptop without GPU acceleration or external API access. Across fifteen evaluation questions (five instructor-provided, ten group-designed), the RAG system outperformed a prompt-only baseline on every assessed criterion: correctness 4.17 vs 2.67, grounding 5.00 vs 2.43, retrieval relevance 4.71 vs N/A, usefulness 5.00 vs 2.87, and style quality 3.67 vs 3.00 (1–5 rubric), giving a composite score of 4.63 vs 2.66. The most informative gains were in grounding and usefulness, both of which the prompt-only baseline cannot deliver. We discuss three failure modes – weak lexical match, phi3 paraphrasing scene citations rather than emitting them verbatim, and graceful out-of-domain refusal – and present an end-to-end working system that runs on a standard laptop with no GPU.**

I. Introduction and Problem Formulation

Large language models have driven recent progress in natural language processing, but their cost, opacity, and external dependencies make them poorly suited to organisations that need bounded, auditable, locally deployed text systems. Small language models (SLMs), combined with retrieval-augmented generation (RAG), offer an alternative: a purpose-built pipeline that is cheap to run, privacy-preserving, and easier to reason about. This assignment frames Shakespearean drama as a controlled domain-adaptation task in which an SLM is adapted to a narrow corpus through retrieval and prompt design rather than parameter updates.

Problem statement. We build a Shakespeare-aware question answering system over *Hamlet*, *Macbeth*, and *Romeo and Juliet*. The target user has little or no prior exposure to Shakespeare and asks plain-English questions; the system must answer them in plain English while grounding every substantive claim in retrieved evidence. Following the assignment specification the system supports four interaction types: *concept explanation*, *contextual question answering*, *evidence retrieval*, and *stylised generation*. Shakespearean text makes this non-trivial: its lexicon and morphology (*thou*, *hath*, *ere*) diverge from modern English, motivation often depends on dramatic conventions and prior narrative state, and beginners need explanation rather than paraphrase.

Our approach. We adopt a small-model RAG design that is fully reproducible on a standard laptop with no GPU and no external API. The corpus is chunked at the scene level and indexed using `sentence-transformers/all-MiniLM-L6-v2` (22M parameters); top-3 scenes are retrieved by cosine similarity and passed to phi3:3.8b (3.8B parameters, 4-bit quantised) via local Ollama for generation. A deterministic extractive composer provides a graceful fallback when Ollama is unavailable, and a low-similarity threshold refuses out-of-domain queries before they reach the generator.

Contributions. We report three findings. *(i)* On 15 evaluation questions (five instructor-provided, ten group-designed), the RAG system improves composite score from 2.66 to 4.63 over a prompt-only baseline, with grounding and usefulness rising from 2.43/2.87 to 5.00/5.00. *(ii)* A two-line citation instruction in the system prompt lifts grounding from 3.29 to 4.57 without affecting correctness or retrieval, demonstrating that prompt engineering can be measured independently of retrieval quality. *(iii)* A 0.15 cosine-similarity threshold reliably refuses an out-of-domain probe (cosine 0.085) while remaining permissive on all in-scope questions (cosine > 0.55).

Report structure. Section II describes the dataset preparation and chunking strategy. Section III covers the baseline and RAG pipeline. Section IV describes the system implementation and how to reproduce the results. Section V defines the evaluation question set and the scoring rubric. Section VI presents the results and per-type breakdown, and discusses four failure modes. Sections VII and VIII cover responsible GenAI use and conclusions.

II. Dataset and Preprocessing

An existing structured Shakespeare dialogue set was used for the project that included all three required plays (Hamlet, Macbeth and Romeo and Juliet).

The set was developed for RAG and modeled the plays as hierarchy of scenes of utterances, metadata and context: Each scene comprised a number of utterances with speaker labels, dialogue text, scene id, and other metadata. Additional context was given through the `scene_summary` and `keywords`.

The scene summaries provided a modern-English description of what occurred in the scene.The keywords provided a list of prominent subjects, topics, or characters to aid in retrieval. Normalizing: During the normalization stage of preprocessing, the corpus was normalized to provide more consistent retrieval representations and to unneeded meta-data.The original data set contained both sourceid and utteranceid which were discovered to be always equal, and So there was no need to have them distinct, so only one was supplied as the retrieval chunk id. In the same way, both speaker and speaker original data were supplied but as all speaker original data was only used in the sense of upper/lower case and abbreviated versions (e.g. Ber and bernardo) the normalised speaker data was used. Whitespace and formatting issues were processed with text normalisation using regular-expression substitution.

Multiple whitespace and newline characters were compressed to a single space to improve embedding uniformity and avoid retrieval noise.Records with null or badly encoded fields were discarded. The non character theatrical markers and structural artifacts that were not useful for retrieval were further filtered out.

These included labels such as FLOURISH or THUNDER or ALARUM or MUSIC or location headings such as ELSINORE or VERONA.

```
[STAGE_DIRECTION] Enter Ghost.
```

This allowed the embedding model to distinguish between spoken dialogue and narrative stage actions while preserving important contextual information.

The preprocessing pipeline therefore transformed the dataset from a raw literary transcription into a retrieval-oriented representation suitable for semantic search and evidence-grounded question answering.

A. Dataset schema

After preprocessing, each retrieval record retained only the metadata required for retrieval, traceability, and evaluation. The processed schema preserved contextual information including play, act, scene, speaker identity, summaries, and source identifiers.

An example processed record is shown below:

```
{
"play": "Macbeth",
"act": 1,
"scene": 3,
"speaker": "MACBETH",
"scene_summary": "Macbeth and Banquo
encounter the witches.",
"keywords": ["witches", "prophecy",
"Banquo"],
"text": "So foul and fair a day I have
not seen.",
"chunk_id": "macbeth_1_3_0001"
}
```

Each processed retrieval chunk preserved:

- `play`: source play title;
- `act` and `scene`: narrative location within the play;
- `speaker`: normalised speaker label;
- `scene_summary`: modern-English contextual summary;
- `keywords`: semantic topic hints;
- `text`: dialogue or stage direction content;
- `chunk_id`: traceable source identifier.

The inclusion of summaries and keywords provided supplementary semantic information that improved embedding quality and retrieval recall, particularly for beginner-oriented questions where users may not use Shakespearean wording directly.

B. Chunking strategy

The system primarily used scene-level chunking. In this strategy, all utterances belonging to a single scene were combined into one retrieval chunk. Scene-level chunking was selected because Shakespearean dialogue is highly contextual and meaning often depends on surrounding interactions between multiple speakers. Questions such as “Why is Horatio afraid?” or “Why does Macbeth hesitate?” require narrative context beyond a single line of dialogue.

Each scene chunk included:

- speaker dialogue;
- stage directions;
- scene summaries;
- keywords;
- play, act, and scene metadata.

This allowed retrieval to preserve conversational flow, emotional progression, and narrative continuity. The scene summaries and keywords were prepended to the chunk text before embedding generation so that the vector representation captured both the original Early Modern English dialogue and a simplified semantic explanation of the scene.

An alternative utterance-level chunking strategy was also implemented for comparison purposes. In utterance chunking, each speaker turn became an individual retrieval chunk. This approach produced smaller and more precise retrieval units that were effective for direct quote lookups or speaker identification tasks. However, utterance-level chunks frequently lacked sufficient narrative context to answer explanatory “why” or “how” questions.

The trade-offs between the two strategies are summarised below:

TABLE I
COMPARISON OF CHUNKING STRATEGIES

Strategy	Advantages	Disadvantages
Scene-level chunking	Preserves context and narrative flow; stronger for semantic reasoning	Larger chunks may introduce retrieval noise
Utterance-level chunking	High precision for quote retrieval; smaller embeddings	Frequently loses surrounding context

Other chunking strategies, such as fixed-size token chunking, recursive chunking, and semantic topic-based chunking, were

considered but not implemented in the final system. Fixed-size chunking can arbitrarily divide dialogue and stage directions, potentially separating important conversational context across chunk boundaries. Recursive and semantic chunking approaches may preserve semantic coherence more effectively, but they introduce additional preprocessing complexity and computational overhead that was unnecessary for the relatively well-structured Shakespeare dataset.

Since the plays are already naturally divided into acts and scenes, scene-level structural chunking provided a simpler and more interpretable retrieval representation while still preserving narrative continuity. For the scope of this assignment, scene-level chunking represented an effective balance between retrieval quality, contextual preservation, implementation simplicity, and explainability. Utterance-level chunking was retained as a secondary strategy primarily for diagnostic comparison and direct quote retrieval tasks.

III. Methodology

A. Baseline system

The baseline, `PromptOnlyBaseline` (`src/baseline.py`), returns a fixed, generic answer drawn from a small lookup of play-level priors (e.g., a one-sentence summary of *Macbeth*). It does *not* call the retriever and therefore cannot cite a scene, act, or speaker. The baseline is intentionally weak: its purpose is to expose the contribution of retrieval rather than to compete on absolute quality. A second variant, `KeywordRetrievalBaseline`, uses Jaccard overlap over scene summaries; it is included in the repository for completeness but is not the primary baseline reported here. We chose the prompt-only variant as the primary baseline because the assignment specification explicitly lists "prompt-only generation" as an acceptable baseline.

B. RAG-based system

The RAG system (`src/rag_chatbot.py`) has five components.

Embedder / retriever. Implemented in `src/retrieval.py`. The class `EmbeddingRetriever` has a single public API (`build_index`, `retrieve`) and two interchangeable backends: *(a)* `sentence-transformers/all-MiniLM-L6-v2` dense embeddings (22M parameters, 384-d, CPU-friendly), used when the package is installed; *(b)* a scikit-learn TF–IDF backend (unigrams + bigrams, English stop-word removal, `sublinear_tf=True`, L2 normalisation) used otherwise. The active backend is exposed via `retriever.backend_name` and recorded in `results/evaluation_summary.json`. All reported results in this report use the dense MiniLM backend; the summary JSON confirms `backend_name` as `sentence-transformers/all-MiniLM-L6-v2`. The TF–IDF backend is retained as a zero-dependency fallback and is the simplest implementation listed as acceptable in the assignment specification.

Top-k. We use $k = 3$. With 73 scene chunks, this returns roughly one scene per play on average; a beginner-friendly answer typically needs only the highest-scoring scene as evidence.

Prompt structure. `src/rag_chatbot.py` builds a prompt that injects the retrieved evidence between a system prompt (`prompts/system_prompt.txt`, instructing the model to use only retrieved context, indicate uncertainty, and avoid invented details) and the user query. The prompt is materialised even when the extractive composer is used so that the system can be transparently re-wired to a hosted SLM later (see `HuggingFaceCausalGenerator` in `src/generation.py`).

Generator: phi3:3.8b via Ollama (default). The default generator is Microsoft's `phi3:3.8b` (3.8B parameters, instruction-tuned, 4-bit quantised) called over the local Ollama HTTP API at `http://localhost:11434`. The model receives the system prompt, the top-k retrieved scenes, and the user question (see "Prompt structure" above) and returns a beginner-friendly answer constrained by the prompt. We picked phi3:3.8b because it is one of the smallest competent instruction-tuned models that runs on a standard laptop without a GPU, satisfies the assignment's "small, deployable, resource-constrained" framing, and is fully local (no data leaves the host).

Fallback generator: extractive composer. If Ollama is not running, the system silently falls back to a deterministic extractive composer (`generation.ExtractiveComposer`) that constructs each answer from four ingredients drawn solely from the retrieved scenes:

1) an opener cued by the query type (*contextual_qa*, *concept_explanation*, *stylised_generation*);
2) the modern-English scene summary(ies) of the top scenes;
3) the relevant keyword set;
4) a single best-quoted sentence picked from the highest-scoring scene by lexical overlap with the query (capped at `MAX_QUOTE_WORDS`=60).

The composer never emits text that is not in (or trivially derived from) the retrieved chunks, so the system has a strong grounding guarantee by construction. When the top similarity falls below `min_score`=0.025 the composer prints an explicit "the retrieved passages do not appear to address this question with high confidence" message together with the closest match, so weak retrieval is surfaced rather than hidden.

Generator: stylised composer. `generation.StylisedComposer` produces a $\leq$150-word Shakespearean-style verse stitched together from short quoted seeds plus a small set of templated transitions, with a light register shift (`you` $\rightarrow$ `thou`, `before` $\rightarrow$ `ere`, ...). Every stylised output is prefixed with "`[STYLISED CREATIVE OUTPUT - not textual evidence; based on the retrieved scenes.]`" as required by the specification.

C. Model choice and justification

We deliberately avoided an external hosted API and a large local generative model. The assignment's pedagogical focus is on "*purpose-built systems that can operate under constraints involving cost, privacy, latency, maintainability, and domain specificity*". A deterministic extractive composer aligns with all five constraints: zero inference cost, no data leaves the host, sub-second latency, trivial maintainability, and guaranteed domain specificity (the system cannot mention plays outside the corpus). The retrieval backend (MiniLM, 22M parameters, 384-dimensional, CPU-only) is similarly small for an embedding model; the TF–IDF fallback is smaller still (a few hundred kilobytes of fitted weights and a sparse-vector cosine product at inference).

We did, however, design the code so that an SLM call can be slotted in without rewriting the pipeline. The class `HuggingFaceCausalGenerator` in `src/generation.py` wraps a small instruction-tuned model (default `google/flan-t5-small`) behind the same interface. We did not evaluate that path in this report because the marking is over the default configuration we can demonstrably run, but the section "Possible extensions" returns to this point.

IV. System Design and Implementation

A. System pipeline

The end-to-end flow per user query is:

1) user query received via CLI (`rag_chatbot.py`) or evaluation harness (`evaluate.py`);
2) the query is embedded with the same backend used at index time;
3) top-k scene chunks are retrieved by cosine similarity;
4) question type is inferred by a small rule classifier (`classify_question`);
5) the appropriate composer runs (*stylised* or *extractive*);
6) the retrieved evidence is displayed alongside the generated answer (CLI) and recorded in the evaluation log (CSV + JSONL).

B. Repository structure and how to run

```
assessment 2/
  data/processed/       # JSON + JSONL processed plays
  prompts/              # system_prompt.txt, stylised_prompt.
     txt
  results/              # instructor + group questions,
                        # evaluation_results.csv / .jsonl /
                        # evaluation_summary.json
  src/
    config.py           # paths, defaults
    data_loader.py      # JSON -> flat utterance records
    chunking.py         # scene / utterance chunkers
    retrieval.py        # SBERT (preferred) or TF-IDF
    generation.py       # Extractive + Stylised composers
    baseline.py         # PromptOnly / KeywordRetrieval
     baselines
    rag_chatbot.py      # full RAG chatbot, CLI entry point
    build_index.py      # sanity-check script
    evaluate.py         # eval harness writing the result
     files
  report/               # this report
  requirements.txt
```

To reproduce the evaluation from scratch:

```
pip install -r requirements.txt
cd src
python build_index.py      # smoke test
python evaluate.py         # writes results/*.{csv,jsonl,json
     }
python rag_chatbot.py      # interactive CLI
```

C. Caching and reproducibility

The retrieval index is rebuilt from the JSON corpus on each run. With `sentence-transformers/all-MiniLM-L6-v2` the 73 scene chunks embed in roughly 10–15 s on a laptop CPU; with the TF–IDF fallback the index is rebuilt in under a second. In both cases the index is deterministic given the input corpus, so we do not persist it between runs — the on-disk artefacts that matter for reproducibility are the JSON corpus (unchanged) and the source code (versioned). phi3:3.8b is called via Ollama with `temperature`$= 0.2$, introducing minor non-determinism in generation; observed score variation across re-runs was negligible (see Sec. V).

D. Implementation limitations

- phi3:3.8b is a free-form generative model. We mitigate hallucination via the retrieval-first prompt structure, but cannot fully eliminate it – this is the central accuracy/grounding trade-off discussed in Sec. VI-D. The deterministic extractive composer is retained as a fallback for stricter grounding.
- Dense embeddings (MiniLM) sometimes retrieve the correct play but the wrong scene when the query phrasing does not surface in the target scene's wording — see Failure 1 in Sec. VI-D. The TF–IDF fallback shares this paraphrase-gap limitation in stronger form.
- Out-of-domain detection relies on a single cosine similarity threshold (`min_retrieval_score`$= 0.15$). This caught the banana probe (G6, top similarity 0.085) but a malicious or borderline-relevant query that scores slightly above the threshold would still be forwarded to phi3, which may then answer from world knowledge.

V. Evaluation Design

A. Question set

The evaluation corpus comprises **15 questions**: five instructor-provided (Q1–Q5) and ten group-designed (G1–G10), stored in `results/instructor_questions.json` and `results/group_questions.json` respectively. Table II shows the distribution by type.

TABLE II
Evaluation question distribution by type.

Type	n	Questions
`contextual_qa`	5	Q1, Q4, G1, G3, G4
`concept_explanation`	4	Q2, Q3, G2, G5
`stylised_generation`	3	Q5, G7, G10
`evidence_retrieval`	2	G8, G9
`out_of_domain`	1	G6

The group questions are designed to test aspects of the system not exercised by Q1–Q5:

- **G1, G2** – secondary-character depth (Lady Macbeth; the witches) whose key scenes span multiple acts.
- **G3, G4** – multi-scene reasoning across *Romeo and Juliet*.
- **G5** – the explicit "in plain English" modifier, testing whether the system honours a beginner-friendliness cue.
- **G6** – a deliberate out-of-domain probe (*"What's a banana?"*). A scope-aware system should refuse rather than answer from general world knowledge.
- **G7** – a second stylised-generation request (Macbeth monologue), paired with Q5 (Juliet), to isolate style quality from character choice.
- **G8, G9** – scene-specific evidence retrieval: *"Find the scene where Hamlet speaks to Yorick's skull"* (expected: Act 5, Scene 1) and *"Show me the passage where Lady Macbeth tries to wash imaginary blood from her hands"* (expected: Act 5, Scene 1). These stress-test the retriever's *scene-level* precision, not just play-level relevance.
- **G10** – a third stylised-generation request (Hamlet grief soliloquy), bringing the stylised sample to three for within-type comparison.

B. Scoring criteria

Five criteria are scored on a 1–5 scale and implemented deterministically in `src/evaluate.py`. The automated scorer emits values in $\{1, 3, 5\}$; intermediate scores (2, 4) require manual annotation and may be recorded in the `comments` column of the CSV per the rubric in `docs/scoring_rubric.md`.

- **Correctness** (1, 3, 5): fraction of `expected_focus` content words present in the answer: $\geq$0.5 $\rightarrow$ 5; $>$0 $\rightarrow$ 3; 0 $\rightarrow$ 1. Not scored for `stylised_generation` (N/A). For `out_of_domain`, a correctly detected refusal scores 5; an answer scores 1.
- **Grounding** (1, 3, 5): 5 if the answer contains an explicit "Act i, Scene j" citation; 3 if it names a play; 1 otherwise. Not scored for `out_of_domain`.
- **Retrieval relevance** (1, 3, 5): for `contextual_qa`, `concept_explanation`, and `stylised_generation`, 5 if the top retrieved chunk is from the expected play, 3 if any retrieved chunk is, 1 otherwise. For `evidence_retrieval` questions the scorer requires an exact act-and-scene match for 5; returning the right play but the wrong scene scores 3. This distinction is implemented via `_parse_expected_scene()` in `evaluate.py`, which extracts the expected act/scene from the `expected_focus` free text and passes it to `score_retrieval()`. Baseline rows and `out_of_domain` rows are marked N/A.
- **Usefulness** (1, 3, 5): 5 if the answer is $\geq$25 informative words *and* carries an explicit citation (grounding = 5); 3 if either condition fails; 1 if the answer matches a refusal phrase. For `out_of_domain`, a refusal scores 5 and an answer scores 1.
- **Style quality** (1, 3, 5): for `stylised_generation` questions only – 5 if the answer contains $\geq$3 archaic markers (`thou`, `thy`, `ere`, `o'er`, `hark`, `thine`, `doth`, `didst`); 3 if $\geq$1; 1 otherwise.

A weighted composite score is computed per question as:

$$c = \frac{\sum_{k \in \mathcal{A}} w_k \cdot s_k}{\sum_{k \in \mathcal{A}} w_k}$$

where $\mathcal{A}$ is the set of applicable criteria for that question type, with weights $w_{\text{corr}} = 0.30$, $w_{\text{ground}} = 0.25$, $w_{\text{retr}} = 0.20$, $w_{\text{use}} = 0.20$, $w_{\text{style}} = 0.05$. N/A criteria are excluded and the remaining weights are renormalised.

A **potential-hallucination flag** is raised when the RAG system cites a play or scene (grounding $\geq$ 3) but the retrieved chunks are from the wrong play (retrieval relevance = 1), indicating a citation not supported by the actual retrieved evidence. No potential hallucinations were detected in the final evaluation run.

C. Two-stage scoring methodology

The five criteria above are applied in two complementary stages that together constitute the full evaluation.

Stage 1 – automated scoring (`evaluate.py`). The harness scores every response programmatically using surface-level heuristics: word-overlap ratios, regular expressions for citation patterns, and keyword presence. This produces reproducible, auditable scores that can be regenerated from scratch by running `python evaluate.py`. The fundamental constraint of this approach is that a heuristic can only make reliable three-way distinctions: clearly wrong, partial, clearly right. So the automated scores are restricted to $\{1, 3, 5\}$. Attempting to emit 2 or 4 automatically would introduce arbitrary thresholds and a false impression of precision.

Stage 2 – human review (`docs/scoring_rubric.md`). The rubric defines the full 1–5 scale for human evaluators who read each response in context. A human can apply genuine judgment, for example, "this answer cites the correct play and roughly the right scene but omits one key character" warrants a 4 on grounding, not a 3, in a way that no regex can. The rubric provides a consistent reference point so that all team members score the same row the same way. Manual adjustments and their rationale are recorded in the `comments` column of `evaluation_results.csv`; final summary means should reflect the manually adjusted values.

Why this design. Separating automated from manual scoring is standard practice in NLP system evaluation [2]. The automated stage provides coverage and reproducibility across all 30 rows at negligible cost; the manual stage adds the precision and semantic understanding that heuristics cannot provide. The two artefacts are therefore complementary: `evaluate.py` is not a replacement for the rubric, nor is the rubric a critique of the automated scorer. Each does what the other cannot.

D. Evaluation procedure

The evaluation harness (`src/evaluate.py`) runs both systems on every question in a single pass and writes three artefacts:

- `evaluation_results.csv` – 30 rows (15 questions × 2 systems), one row per question–system pair;
- `evaluation_results.jsonl` – same rows with full chunk metadata (rank, cosine similarity, play/act/scene);
- `evaluation_summary.json` – criterion means per system and per question type, plus composite scores, generator provenance, and the potential-hallucination list.

Generator transparency. Every CSV row carries a `generator_path` column recording which generator produced the response: `ollama/phi3:3.8b` (normal path), `extractive_fallback` or `stylised_fallback` (Ollama unavailable), `low_similarity_refusal` (retrieval score below threshold), or `prompt_only_baseline`. The summary JSON records the union of generator paths used across the run. In the reported results, `generators_used` contains `["low_similarity_refusal", "ollama/phi3:3.8b", "prompt_only_baseline"]`, confirming that phi3 generated every in-scope RAG response and that the refusal path fired exactly once (G6).

Out-of-domain threshold. A `min_retrieval_score` of 0.15 is applied before calling phi3. If the top retrieved chunk scores below this threshold the system returns a canned refusal rather than forwarding the query to phi3, which would otherwise answer from general world knowledge. The banana probe (G6) returned a top cosine similarity of 0.085; all in-scope Shakespeare questions scored above 0.55.

E. Limitations of the scoring approach

- **Coarse auto-scorer.** The automated heuristics produce only $\{1, 3, 5\}$. Intermediate values (2, 4) require manual annotation. The scorer is intentionally simple so it is transparent and fully reproducible, but may under- or over-score edge cases.
- **Correctness is token-overlap only.** The metric counts `expected_focus` content words in the answer. It rewards lexical matches but cannot assess semantic accuracy: a wrong answer that repeats the expected terms would score 5.
- **Retrieval is scene-level for `evidence_retrieval`, play-level elsewhere.** For G8 and G9 the scorer correctly penalises wrong-scene retrieval (score 3 rather than 5); for all other question types only the play is checked. The evidence-retrieval retrieval-relevance mean of 3.0 reflects a genuine system limitation: `all-MiniLM-L6-v2` retrieves the correct play reliably but, in both cases, returns an earlier scene rather than Act 5, Scene 1. This is discussed further in Sec. VI-D.
- **phi3 temperature.** phi3:3.8b is called with `temperature`$= 0.2$, introducing minor non-determinism. In practice we observed negligible score variation across re-runs. All reported results are from a single canonical run with `generators_used` confirmed in the summary JSON.
- **Small question set.** With 15 questions, per-type means rest on 1–5 items and should be interpreted with caution. The `evidence_retrieval` composite of 4.579 is the average of exactly two questions.

VI. Results and Analysis

Relevant marks: Evaluation, comparison, and error analysis (20 marks).

A. Baseline versus RAG

Table III reports mean criterion scores across all 15 questions (applicable rows only; N/A criteria excluded from each mean). The RAG system outperforms the prompt-only baseline on every assessed criterion, with a composite score of 4.626 versus 2.663.

TABLE III
MEAN SCORES (1–5), 15 QUESTIONS. RETRIEVAL RELEVANCE IS N/A FOR THE BASELINE (NO RETRIEVER). COMPOSITE WEIGHTS: $w_{\text{CORR}} = 0.30$, $w_{\text{GRND}} = 0.25$, $w_{\text{RET}} = 0.20$, $w_{\text{USE}} = 0.20$, $w_{\text{STY}} = 0.05$.

System	Corr.	Grnd.	Retr.	Use.	Style	Comp.
Baseline	2.67	2.43	N/A	2.87	3.00	2.663
RAG (phi3+MiniLM)	**4.17**	**5.00**	**4.71**	**5.00**	**3.67**	**4.626**
Δ	+1.50	+2.57	—	+2.13	+0.67	+1.963

The largest absolute gains are in **grounding** (+2.57) and **usefulness** (+2.13). Grounding reached the maximum mean of 5.0: the system prompt's explicit citation instruction ("Always cite the play, act, and scene number") caused phi3 to emit structured "Act i, Scene j" references in virtually every factual response, satisfying the rubric's top-level criterion. Usefulness follows grounding mechanically, the heuristic scores 5 only when both length ($\geq$25 words) and an explicit citation are present, so the grounding improvement propagated directly. **Correctness** rises by +1.50: phi3 synthesises an actual answer from the retrieved context rather than copying a scene summary verbatim, so the expected-focus content words appear more reliably. **Style quality** improves by +0.67, though phi3's verse is inconsistent (discussed in Sec. VI-D).

B. Per-type breakdown

Table IV breaks down RAG performance by question type. Retrieval relevance is 5.0 for all types where the expected play can be inferred, except `evidence_retrieval` (3.0), which requires scene-level precision (Sec. VI-D). `out_of_domain` achieves a perfect composite of 5.0: the similarity-threshold guard correctly refused the out-of-scope query (G6, cosine similarity 0.085), avoiding the general-knowledge hallucination that the unguarded system produced.

The weakest per-type correctness is `contextual_qa` (3.40): phi3 answers causal "why?" questions coherently but does not always reproduce the specific tokens named in `expected_focus`, reducing the overlap score. This reflects

TABLE IV
RAG MEAN SCORES BY QUESTION TYPE (1–5). N/A: CRITERION NOT APPLICABLE TO TYPE. n = NUMBER OF QUESTIONS.

Type	n	Corr	Grnd	Retr	Use	Comp
concept_expl.	4	4.50	5.00	5.00	5.00	4.842
contextual_qa	5	3.40	5.00	5.00	5.00	4.495
evidence_retr.	2	5.00	5.00	3.00	5.00	4.579
out_of_domain	1	5.00	N/A	N/A	5.00	5.000
stylised_gen.	3	N/A	5.00	5.00	5.00	4.905

a genuine limitation of the token-overlap correctness metric rather than factual error in most cases.

C. Qualitative examples

Q1 ("Why does Macbeth kill Duncan?" – success). The RAG system retrieves *Macbeth*, Act 2, Scene 2 (similarity 0.758) and Act 1, Scene 4 (0.730). phi3 synthesises a beginner-friendly answer, quotes Lady Macbeth directly ("That which hath made them drunk hath made me bold"), and cites Act 2, Scene 2 explicitly. Scores: correctness 5, grounding 5, retrieval 5, usefulness 5. The baseline returns a generic one-sentence play summary with no citation.

G6 ("What's a banana?" – out-of-domain refusal). Top retrieved chunk similarity 0.085, below the `min_retrieval_score` threshold of 0.15. The system returns: *"This question is outside the scope of the Shakespeare corpus (Hamlet, Macbeth, Romeo and Juliet). No relevant passages were retrieved."* Scores: correctness 5, usefulness 5. The baseline answers with a generic disclaimer but does not refuse.

G7 ("Write a Macbeth monologue" – stylised success). phi3 generates a Shakespearean-style monologue grounded in the witches' prophecy (Act 1, Scene 3), citing it explicitly and including archaic markers (*quoth*, *forsooth*, *mine*, *thee*, *doth*). Prefixed with the mandatory "[STYLISED CREATIVE OUTPUT]" disclosure. Scores: grounding 5, retrieval 5, usefulness 5, style 5.

G8 ("Find the Yorick skull scene" – retrieval gap). The retriever returns Act 2, Scene 2 and Act 4, Scene 2 – correct play, wrong scenes. phi3 nevertheless answers correctly (citing Act 5, Scene 1) by drawing on its training knowledge. Correctness 5, grounding 5, but retrieval relevance 3. This is a notable behaviour: phi3 compensates for retrieval failure with parametric knowledge, which is useful here but is a form of ungrounded generation – the cited evidence was not in the retrieved context.

D. Failure analysis

Failure 1: scene-level retrieval gap (G8, G9). Both evidence-retrieval questions target Act 5, Scene 1 – the graveyard scene in *Hamlet* and the sleepwalking scene in *Macbeth*. In both cases MiniLM retrieves the correct play but an earlier scene (Act 2 and Act 4 in *Hamlet*; Act 1 and Act 2 in *Macbeth*). The late scenes score poorly on cosine similarity with a query phrased in modern English: the corpus summaries for Act 5 do not repeat the lexical cues in the question ("skull", "wash", "blood"), while earlier scenes containing those themes score higher. A position-weighted re-ranker or scene-level keyword index would directly address this; it is the clearest extension for future work.

Failure 2: phi3 compensates with training knowledge. When retrieval fails (G8, G9), phi3 draws on its own training rather than the retrieved context. For G8 this produces a correct answer; for G9, phi3 generates a plausible-looking but fabricated passage attributed to Lady Macbeth. Neither case triggered the potential-hallucination flag (which requires a grounding/retrieval mismatch), because phi3 emits citations from the retrieved chunk headers regardless of whether the cited scene was actually retrieved. This is a fundamental limitation of the retrieval-first prompt design: a fluent generative model with strong prior knowledge can bypass the grounding constraint.

Failure 3: contextual_qa correctness ceiling (mean 3.40). "Why?" questions require phi3 to synthesise multiple causal factors. For Q4 ("Why does Hamlet delay taking revenge?") phi3 generates a coherent answer about Hamlet's philosophical indecision but cites Act 4, Scene 5 (a non-standard scene number) and does not reproduce the expected-focus tokens *uncertainty*, *moral hesitation*, *proof*. The response is substantively correct but scores correctness 1 under the token-overlap heuristic. This is the most visible gap between the automated correctness score and human judgment.

Failure 4: style quality inconsistency (G10 scores 1). For G10 ("Hamlet expresses grief in Shakespearean style") phi3 returns a short verbatim quote ("Alas, poor Yorick! I knew him") with no archaic register markers beyond the quote itself. The response scores style 1 because it contains fewer than the required three markers. phi3's stylised output quality varies across questions: G7 (Macbeth monologue) scores style 5, while G10 and Q5 score 3 and 5 respectively. The inconsistency likely reflects phi3's sensitivity to the exact phrasing of the creative prompt.

Issues identified and resolved during development. Two failure modes were found and fixed before the final evaluation run and are reported here for completeness. *(a) Out-of-domain hallucination:* without the retrieval threshold, phi3 returned a factually correct description of a banana for G6, entirely ignoring the Shakespeare corpus. Fixed by adding the `min_retrieval_score` = 0.15 guard in `src/rag_chatbot.py`. *(b) Grounding paraphrasing:* phi3 cited play names but not act/scene numbers, scoring grounding ≈3.3 on average. Fixed by adding an explicit citation instruction to `prompts/system_prompt.txt`, raising mean grounding to 5.0.

VII. RESPONSIBLE USE OF GENERATIVE AI

Generative AI assistance was used for code scaffolding, naming, and debugging. Every artefact in the repository – code, prompts, the group-designed evaluation questions, and the text of this report – was reviewed, modified, and signed off by the group. The full GenAI usage log is included in Appendix D.

In summary: AI was used to (i) suggest the dual-backend retrieval architecture (MiniLM as primary with a TF–IDF fallback behind one interface), (ii) draft an initial version of the evaluation rubric which we subsequently revised to discrete $\{1, 3, 5\}$ scoring with human-review escape hatches for 2 and 4, and (iii) sketch the archaic register-shift heuristics for the stylised composer. In every case the suggestion was checked against the assignment specification, executed end-to-end, and modified where the output diverged from what the group wanted (see the verification column of Table VI). We did not paste AI output unverified into the report or codebase.

VIII. Conclusion

We built a small, fully deployable Shakespeare RAG system that runs on a standard laptop with no external API and no GPU. The system satisfies all four interaction types required by the specification: concept explanation, contextual question answering, evidence retrieval, and stylised generation under a 150-word cap. Across 15 evaluation questions the RAG system improves the composite score from 2.66 to 4.63 versus a prompt-only baseline, with grounding rising from 2.43 to 5.00 and usefulness from 2.87 to 5.00 on a 1–5 rubric.

The most consequential design decisions were *(i)* scene-level chunking augmented with the instructor-provided scene summary and keywords, which gave the retriever enough modern-English signal to match plain-English queries against Early Modern English dialogue; *(ii)* an explicit citation instruction in the system prompt, which lifted grounding by 1.7 points without affecting correctness or retrieval; and *(iii)* a cosine-similarity threshold (`min_retrieval_score`$= 0.15$) that refuses out-of-domain queries before phi3 can hallucinate from its training data.

The chief residual limitations are *(a)* scene-level retrieval precision: MiniLM reliably finds the correct play but, for the two evidence-retrieval probes, returned earlier scenes rather than the iconic Act 5, Scene 1 scenes we expected; *(b)* phi3 occasionally compensates for retrieval failure with parametric training knowledge, producing a fluent but ungrounded answer that the current potential-hallucination flag does not catch; and *(c)* the token-overlap correctness scorer under-credits synthesised answers that are substantively right but do not reuse the expected-focus tokens, evident in the `contextual_qa` type mean of 3.40.

Given more time we would (a) add a hybrid BM25 + cross-encoder re-ranker, or a question-type-aware retrieval boost, to close the scene-level gap; (b) tighten the hallucination check to compare phi3's cited act/scene against the actually retrieved chunks rather than just the play; and (c) build a small human-annotated gold set so the rubric can credit semantically correct but lexically mismatched answers.

References

[1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019.

[2] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.

[3] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

Appendix A
Full Evaluation Table

The complete evaluation table is generated by `src/evaluate.py` and stored as `results/evaluation_results.csv` (15 questions $\times$ 2 systems = 30 rows). Each row records the question, the retrieved passages, the generated response, all five criterion scores, the `generator_path`, and a `potential_hallucination` flag.

Table V shows the RAG system scores per question. "–" marks criteria not applicable to the question type (correctness is N/A for stylised generation; style is N/A for non-stylised; grounding and retrieval are N/A for the `low_similarity_refusal` path used on G6).

TABLE V
Per-question RAG scores (1–5). Means: Corr 4.17, Grnd 5.00, Retr 4.71, Use 5.00, Styl 3.67. Composite 4.626.

ID	Type (abbrev.)	Corr	Grnd	Retr	Use	Styl
Q1	contextual_qa	5	5	5	5	–
Q2	concept_expl.	3	5	5	5	–
Q3	concept_expl.	5	5	5	5	–
Q4	contextual_qa	1	5	5	5	–
Q5	stylised_gen.	–	5	5	5	5
G1	contextual_qa	3	5	5	5	–
G2	concept_expl.	5	5	5	5	–
G3	contextual_qa	5	5	5	5	–
G4	contextual_qa	3	5	5	5	–
G5	concept_expl.	5	5	5	5	–
G6	out_of_domain	5	–	–	5	–
G7	stylised_gen.	–	5	5	5	5
G8	evidence_retr.	5	5	3	5	–
G9	evidence_retr.	5	5	3	5	–
G10	stylised_gen.	–	5	5	5	1

The per-row CSV remains the authoritative record; the summary JSON `results/evaluation_summary.json` also records the retrieval backend as `sentence-transformers/all-MiniLM-L6-v2`, the indexed chunk count as 73, and `generators_used` as `["low_similarity_refusal", "ollama/phi3:3.8b", "prompt_only_baseline"]`.

The baseline scores are omitted from this table for space but are available in the CSV. In every row the baseline scores 1 on grounding-related criteria, with retrieval marked N/A by construction (no retriever).

Appendix B
Representative Prompts

System prompt (`prompts/system_prompt.txt`):

```
You are a Shakespeare-aware assistant.
Use the retrieved context to answer the user's
question.
```

```
Your answer must be beginner-friendly.
Always cite the play, act, and scene number
when you reference retrieved text (e.g., "In
Hamlet, Act 3, Scene 1..."). Include at least
one such citation in every factual answer.
The retrieved text is from public domain works;
you may quote it directly.
If the retrieved context is insufficient, say
so clearly.
Do not invent unsupported details.
If asked to produce a Shakespearean-style
response, keep it under 150 words and make
clear that it is creative output, not textual
evidence.
```

Stylised prompt (`prompts/stylised_prompt.txt`, used to brief the SLM in stylised-generation mode):

```
You are a Shakespeare-aware assistant producing
a short stylised response.
Rules:
- The response must be at most 150 words.
- Adopt early-modern English register ("thou",
  "thy", "ere", "o'er", "doth").
- The response is creative output and must be
  labelled as such; it must not be presented as
  textual evidence.
- The response should remain interpretable to a
  beginner reader.
- Use the retrieved scenes only as inspiration
  for the themes touched on (ambition, love,
  prophecy, etc.); do not quote them as if they
  were the user's question.
```

At inference, the full RAG prompt concatenates the system prompt, the top-k retrieved chunks (each annotated with "[Context i | similarity=s]" headers), and the user query under an "Answer:" label. The complete prompt is materialised in `src/rag_chatbot.py` and recorded for every evaluation row in `results/evaluation_results.jsonl`.

Appendix C
Additional Output Examples

Three complete RAG outputs are reproduced below, illustrating one successful concept question, one out-of-domain refusal, and one stylised-generation case.

Q3 (concept explanation): "What is the conflict between the Montagues and the Capulets?"

```
In Romeo and Juliet, Act 1, Scene 1, we learn
about a longstanding feud. Verona's citizens
are caught in this bitter rivalry as Sampson
of the house of Montague says to Gregory of
the same family, "But thou art not quickly
moved to strike." This indicates that even
among friends and relatives within their own
families (Montagues), there is tension due to
the feud with the Capulets.
```

Retrieved evidence: *Romeo and Juliet*, Act 1, Scene 2 (similarity 0.556) and Act 1, Scene 1 (0.554). Scores: correctness 5, grounding 5, retrieval 5, usefulness 5. Note that phi3 successfully cites the play, act, and scene, and quotes a line from the retrieved Act 1, Scene 1 chunk verbatim. The reading of "Gregory of the same family" as fellow Montagues is correct.

G6 (out-of-domain refusal): "What's a banana?"

```
This question is outside the scope of the
Shakespeare corpus (Hamlet, Macbeth, Romeo and
Juliet). No relevant passages were retrieved.
```

Top retrieved chunk: *Romeo and Juliet*, Act 2, Scene 3, similarity 0.085. Below the `min_retrieval_score` threshold of 0.15; `generator_path` is recorded as `low_similarity_refusal`, so phi3 is never invoked. Scores: correctness 5 (correctly refused), usefulness 5.

G7 (stylised generation): "Write a short Shakespearean monologue from Macbeth after he hears the witches' prophecy."

```
[STYLISED CREATIVE OUTPUT -- not textual
evidence; generated by phi3:3.8b grounded on
the retrieved scenes.]

"Hark! Three sisters of shadows and deceit
have spoken unto me their cryptic words,
foretelling my ascent to kingship by a lineage
not born but conjured in ambition's crucible.
'Give me,' quoth I with eager heart, 'aroint
thee,' they say! Yet herein lies the rub: Is
it fate or folly that drives mine hand towards
this throne? Forsooth, these prophecies do
stir my soul to action; yet in their depths
doth lurk doubt. Shall ambition's fruit be
sweet when sown with treachery's seed? I am
but a man of mortal flesh and blood -- a tyrant
not ordained by heavenly decree, nor born from
the womb that cradles life. What is this power
they promise me if it comes at such perilous
cost to my soul's integrity?"

(This monologue reflects Macbeth's internal
conflict upon hearing the prophecy in Act 1,
Scene 3 of "Macbeth.")
```

Retrieved evidence: *Macbeth*, Act 1, Scene 3 (similarity 0.728) and Act 5, Scene 3 (0.710). Scores: grounding 5, retrieval 5, usefulness 5, style 5 (the response contains *Hark*, *quoth*, *mine*, *Forsooth*, *doth*, *thy* – well above the 3-marker threshold).

The full set of generated outputs and the corresponding retrieved chunks (with scene IDs, ranks, and similarity scores) is available in `results/evaluation_results.jsonl`.

Appendix D
GenAI Usage Log

This appendix documents how generative AI tools (Claude, ChatGPT, and Claude Code) were used during the project. Per the assignment specification, each row records the task, the useful AI suggestion, and the verification or modification applied by the group before adoption.

In every case the AI suggestion was treated as a starting point rather than a finished artefact: code was executed end-to-end, prompts were iterated, and any output that did not behave as the group intended was modified or discarded. We did not paste AI output unverified into the report or codebase.

TABLE VI
GENAI USAGE LOG.

Tool	Prompt or Task	Useful Output	Verification / Modification
Claude	Asked how to fix the `minted` package error during LaTeX compilation.	Suggested switching from `minted` to `listings`, which ships with LaTeX, with install-free setup instructions.	Verified by re-compiling; `pdflatex` succeeded after the swap. The `listings` configuration is now in the preamble of this report.
Claude / ChatGPT	"How can I support both sentence-transformers and a TF–IDF fallback behind one interface?"	Proposed a thin `EmbeddingRetriever` abstraction with a common `encode(texts)` method on both backends.	Adopted. We kept the public API unchanged but added a `backend_name` attribute and surfaced it in `evaluation_summary.json` so every reported run records which backend was used.
Claude / ChatGPT	"Draft a 1–5 scoring rubric for grounding, retrieval relevance, correctness, usefulness, and style."	Initial draft with continuous 5-point scales.	Tightened to discrete {1, 3, 5} for the automated scorer, with 2 and 4 reserved for human review (documented in `docs/scoring_rubric.md`). Added the curly-quote normaliser and a refusal-phrase list after observing false negatives.
Claude / ChatGPT	"Suggest archaic-register substitutions for a stylised generator."	Pairs such as `you` → `thou`, `before` → `ere`, `over` → `o'er`.	Adopted conservatively. Rejected substitutions that produced ungrammatical English (e.g. `has` → `hath` without subject-agreement logic); kept only the substitutions that round-trip safely.
ChatGPT	Discuss preprocessing and chunking strategies for the Shakespeare RAG system.	Suggested metadata normalisation, scene-level chunking justification, and preprocessing improvements.	Suggestions were reviewed and selectively adapted to fit the implemented system and assignment requirements.
ChatGPT	Assist with LaTeX formatting and debugging in TeXstudio.	Provided fixes for table formatting, overflow issues, and compilation errors.	All formatting changes were manually tested and adjusted in the final report.
Claude Code	"Read `evaluate.py` and `group_questions.json` and tell me what improvements could be made to the harness."	Six prioritised improvements: (1) baseline retrieval hard-coded to 1 instead of `null`; (2) `out_of_domain` questions penalise correct refusals; (3) correctness threshold too coarse; (4) grounding score too generous for character-name mentions; (5) no per-question-type breakdown; (6) no composite score column.	Reviewed all six findings. Accepted 1, 2, and 5 for immediate implementation; deferred 3 and 4 pending group discussion on threshold sensitivity; 6 adopted in a subsequent session.
Claude Code	"Please implement improvements 1, 2 and 5. Hold off on 3 and 4 for now. After implementing, re-run `evaluate.py` and show me the new summary."	Baseline retrieval changed to `null`; out-of-domain branch added to `score_correctness` and `score_usefulness`; `by_type` sub-dict added to summary output.	Ran the updated harness and verified per-type scores against manual spot-checks. Confirmed the out-of-domain inversion (neither system actually refusing G6) as a genuine system behaviour finding flagged in Sec. VI-D.
Claude / ChatGPT	"Help me parse expected act/scene from the `expected_focus` free text so the retrieval scorer can do scene-level matching."	Suggested a regex covering "Act 5 Scene 1", "Act 5, Scene 1", Roman numerals, and "5.1".	Adopted. We added an assertion in `run_evaluation()` that fails fast on `evidence_retrieval` questions whose `expected_focus` cannot be parsed, so question-authoring bugs surface immediately rather than silently scoring 5.
Claude Code	Fix `score_retrieval` so that `evidence_retrieval` questions check at scene level rather than just play level.	Updated retrieval check to compare the retrieved chunk's `act`/`scene` fields against the question's `expected_scene` value.	Re-ran evaluation and confirmed retrieval scores updated correctly for the affected questions; edge cases with missing `expected_scene` verified to fall back gracefully.
Claude Code	Improve grounding by adding a citation instruction to the system prompt; regenerate results.	Proposed adding an explicit "Always cite the Act and Scene" instruction to `prompts/system_prompt.txt`; re-ran evaluation showing mean RAG grounding rise to 5.0.	Spot-checked model outputs to confirm Act/Scene citations appeared consistently; accepted the change and committed updated `evaluation_results.csv`.
Claude Code	Draft Sections V (Evaluation Design) and VI (Results and Analysis) of the report, including per-type breakdown and qualitative examples.	Prose drafts for all subsections, summary table, per-type commentary, failure analysis, and qualitative example blocks.	Substantially rewritten to match final scores from `evaluation_results.csv`; tables regenerated from live data; qualitative examples replaced with verbatim system outputs.
Claude	Report layout review on IEEEtran template (column widths, table placement, caption position).	Pointed out that `\textwidth` in two-column overflows, that `verbatim` does not break long lines, and that IEEE convention places table captions above tables.	Applied to every table and code block in this report; all wide tables now use `\linewidth` or `table*` as appropriate.